

Отчёт о проделанной работе по проекту llmq-triton

Александр Андреев

Михаил Маслов

Артём Матвеев

31 марта 2026 г.

Краткое описание

Проект посвящён int4-квантизации весов линейных слоёв и реализации матричного умножения формата W4A16 (активации bf16, веса int4) на Triton. Размерности экспериментов ориентированы на слои модели Llama-3.2-1B-Instruct.

Состав команды

- Александр Андреев
- Михаил Маслов
- Артём Матвеев

Что сделано (первые эксперименты)

1. Реализована построчная симметричная int4-квантизация весов fp16/bf16 с группировкой по оси K (по умолчанию `group_size=128`) и упаковкой двух значений int4 в один байт.
2. Реализован Triton-кernels матмул/линейного слоя для вычисления $X_{\text{bf16}} \cdot W_{\text{int4}}^T$ с распаковкой весов и применением scale на лету.
3. Подготовлен воспроизводимый бенчмарк, сравнивающий скорость с `torch.matmul` (bf16/cuBLAS) на формах матриц, соответствующих проекциям Llama-3.2-1B.

Результаты экспериментов (кратко)

По результатам первых замеров на A100 библиотечный matmul bf16 оказался заметно быстрее текущей реализации int4 по latency на выбранных размерах; при этом int4 даёт выигрыш по памяти весов (порядка $\sim 3.7\times$ с учётом scales при `group_size=128`). Детальные таблицы замеров и окружение приведены в файле `REPORT.md` в репозитории.

Таблица результатов (из REPORT.md)

Слой	W (N×K)	M	t_{bf16} (ms)	t_{int4} (ms)	speedup
q_proj	2048×2048	128	0.015	0.260	0.06
q_proj	2048×2048	512	0.032	0.749	0.04
q_proj	2048×2048	2048	0.111	2.527	0.04
k_proj	512×2048	128	0.014	0.257	0.06
k_proj	512×2048	512	0.014	0.262	0.06
k_proj	512×2048	2048	0.032	0.765	0.04
gate_proj	8192×2048	128	0.035	0.757	0.05
gate_proj	8192×2048	512	0.113	2.521	0.04
gate_proj	8192×2048	2048	0.281	9.660	0.03
down_proj	2048×8192	128	0.041	1.180	0.03
down_proj	2048×8192	512	0.090	3.599	0.03
down_proj	2048×8192	2048	0.333	12.704	0.03

Таблица 1: Время вычисления XW^T : bf16 (cuBLAS) vs int4 (кernels проекта). Полный машиночитаемый вывод: `results/bench_llama.json`.

План выполнения проекта (следующие этапы)

1. Сделать обёртку `QuantizedLinear` для хранения `packed/scales` и вызова W4A16-кernels (с `fallback`-путём для отладки).
2. Подменить линейные слои в `LlamaForCausalLM` (q/k/v/o и MLP: gate/up/down) на квантизованные аналоги.
3. Организовать загрузку и/или конвертацию весов HF: оффлайн-конвертер или `lazy-quantize`, плюс сохранение квантизованного чекпойнта.
4. Провести оценку качества (перплексия) на `wikitext2` при одинаковом препроцессинге.
5. Выполнить сквозной бенчмарк инференса: `latency/token` и память для контекстов 128/512/2048 (`batch=1`).

Ссылка на GitHub

Репозиторий с кодом и отчётом: <https://github.com/Alex-Andrv/project>

Примечание: отчёт с подробными результатами и планом находится в файле `REPORT.md` в корне репозитория.